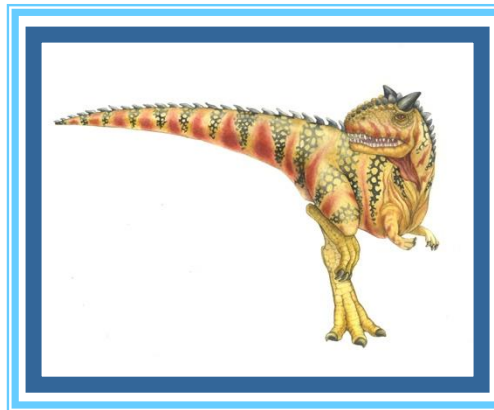


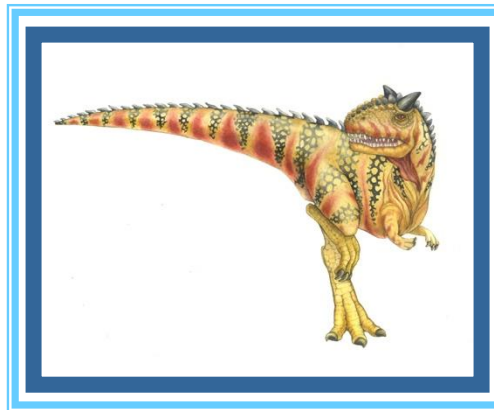
These slides have been modified from the original



A huge thanks to the
authors of Operating
System Concepts for
providing this resource



These slides have been modified from the original



An indicator on a slide means I've modified something on the slide from the original slides

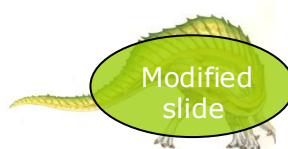
Modified slide

Processes & Threads



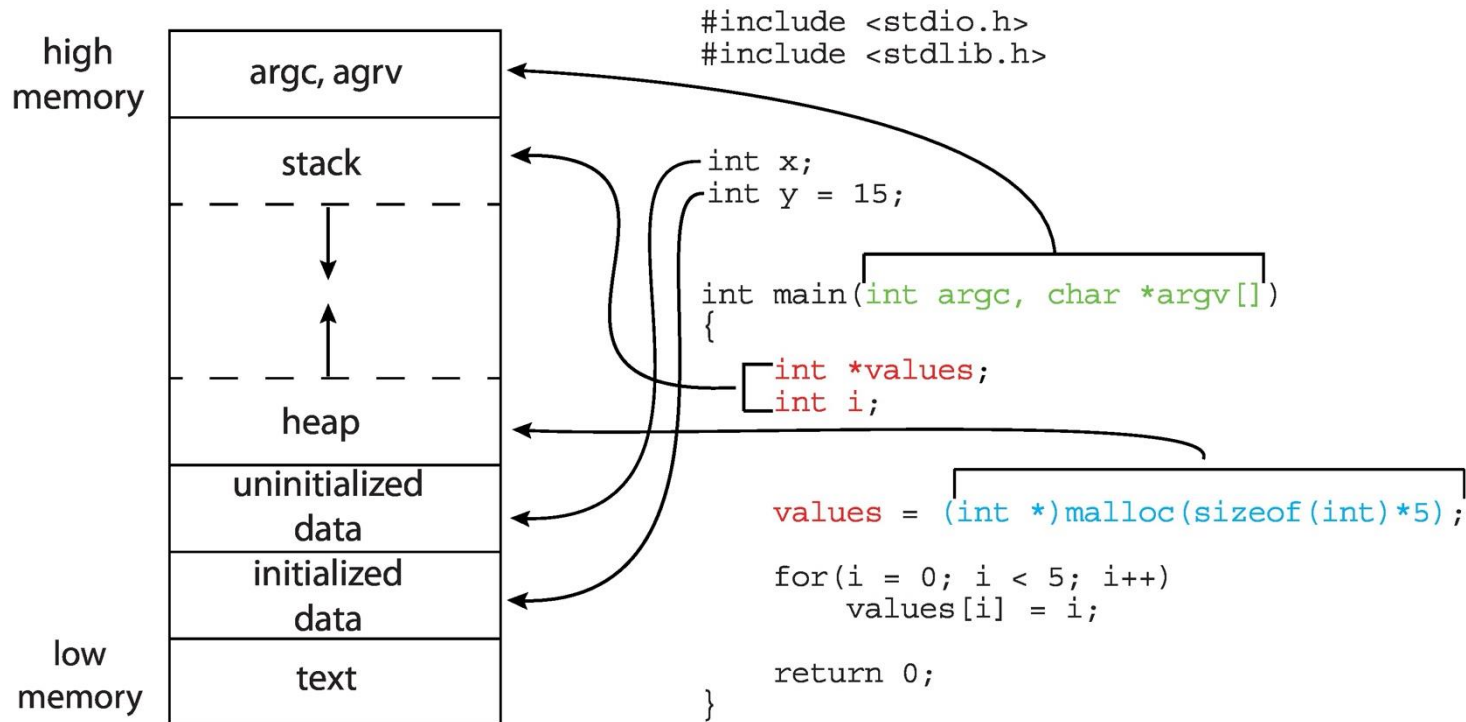
Process Concept

- An operating system executes a variety of programs that run as processes.
- **Process** – a program in execution; process execution progresses in **sequential** fashion i.e. **No parallel execution of instructions of a single process.**
- Multiple parts (**a process in memory**)
 - The program code, also called **text section**
 - Current activity including **program counter**, processor registers
 - **Stack** containing temporary data
 - ▶ Function parameters, return addresses, local variables
 - **Data section** containing global variables
 - **Heap** containing memory dynamically allocated during run time





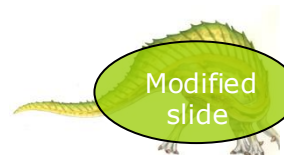
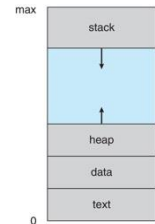
Memory Layout of a C Program





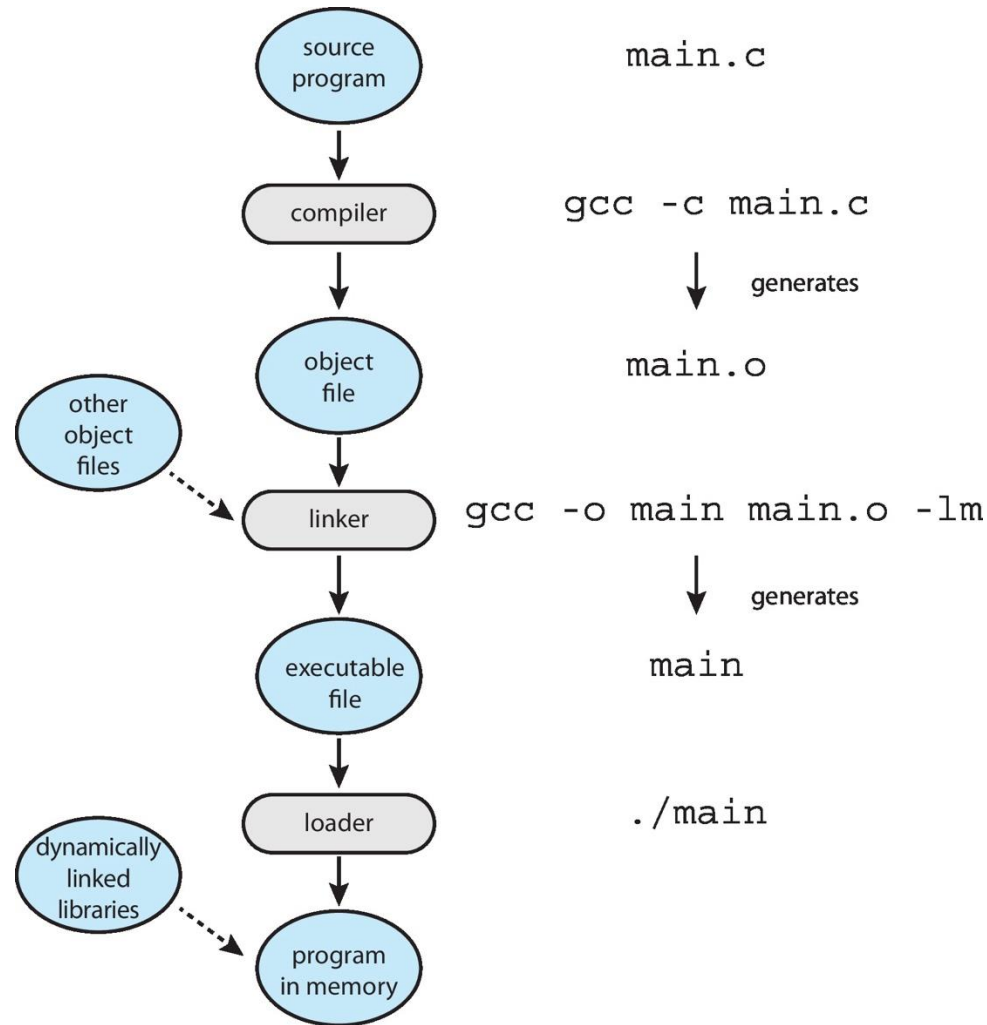
Process Concept (Cont.)

- Program is **passive** entity stored on disk (**executable file**); process is **active**
 - Program becomes a process when an executable file is loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc. e.g. `./run_my_c_program`
- One program can be several processes
 - Consider multiple users executing the same program



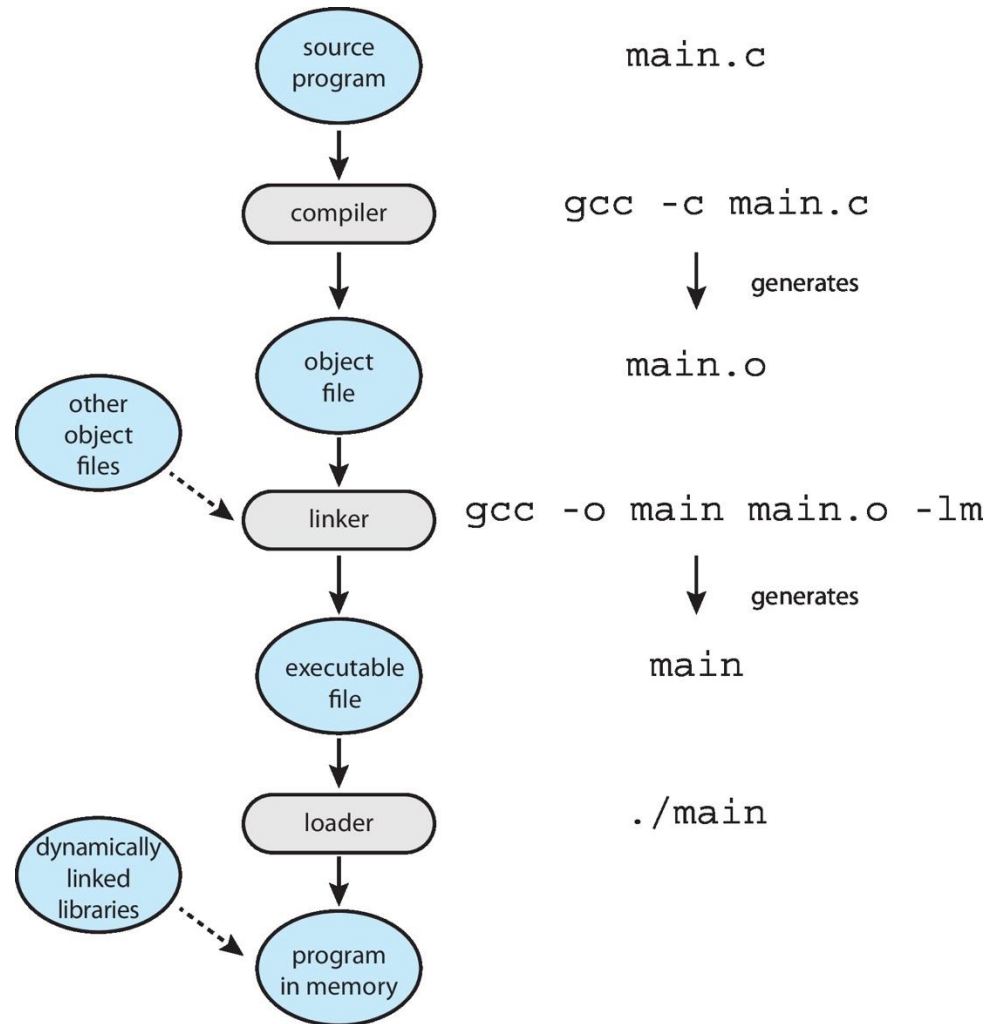


The Role of the Linker and Loader





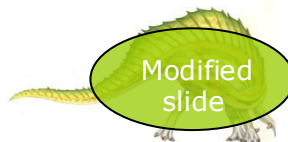
The Role of the Linker and Loader





Linkers and Loaders

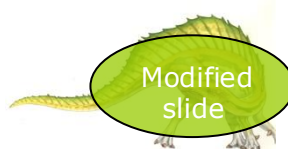
- Source code is compiled into object files (partially linked artefacts) – **relocatable object file**
- **Linker** combines these into single binary **executable** file
 - Also brings in libraries
- Program resides on secondary storage as binary executable
- Must be brought into memory by **loader** to be executed
 - **Relocation** updates address references so code and data work correctly at their assigned **virtual addresses**
- **But** modern general purpose systems don't always link libraries into executables
 - Rather, **dynamically linked libraries** (in Windows, **DLLs**) are loaded as needed, shared by all that use the same version of that same library (loaded once)
- Object, executable files have standard formats, so operating system knows how to load and start them





Why Applications are Operating System Specific

- Apps compiled on one system usually not executable on other operating systems
- **Each operating system provides its own unique system calls**
 - Own file formats, etc.
- Apps can be multi-operating system
 - Written in interpreted language like Python, Ruby, and interpreter available on multiple operating systems
 - App written in language that includes a VM containing the running app (like Java)
 - Use standard language (like C), compile separately on each operating system to run on each
- **Application Binary Interface (ABI)** is low-level equivalent of API, defines how different components of binary code can interface for a given operating system on a given architecture, CPU, etc.



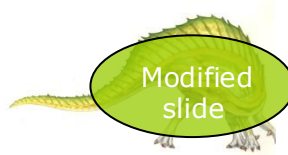


Process State

- As a process executes, it changes **state**
 - **New**: The process is being created
 - **Running**: Instructions are being executed
 - **Waiting**: The process is waiting for some event to occur
 - **Ready**: The process is waiting to be assigned to a processor
 - **Terminated**: The process has finished execution

```
129 static const char * const task_state_array[] = {
130
131     /* states in TASK_REPORT: */
132     "R (running)",      /* 0x00 */
133     "S (sleeping)",     /* 0x01 */
134     "D (disk sleep)",   /* 0x02 */
135     "T (stopped)",      /* 0x04 */
136     "t (tracing stop)", /* 0x08 */
137     "X (dead)",         /* 0x10 */
138     "Z (zombie)",       /* 0x20 */
139     "P (parked)",       /* 0x40 */
140
141     /* states beyond TASK_REPORT: */
142     "I (idle)",         /* 0x80 */
143 };
144
```

From:
<https://github.com/torvalds/linux/blob/3950e975431bc914f7e81b8f2a2dbdf2064acb0f/fs/proc/array.c#L129-L143>





Process State

- As a process executes, it changes **state**
 - **New**: The process is being created
 - **Running**: Instructions are being executed
 - **Waiting**: The process is waiting for some event to occur
 - **Ready**: The process is waiting to be assigned to a processor
 - **Terminated**: The process has finished execution

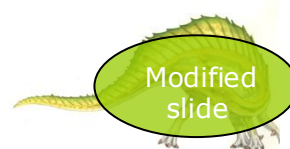
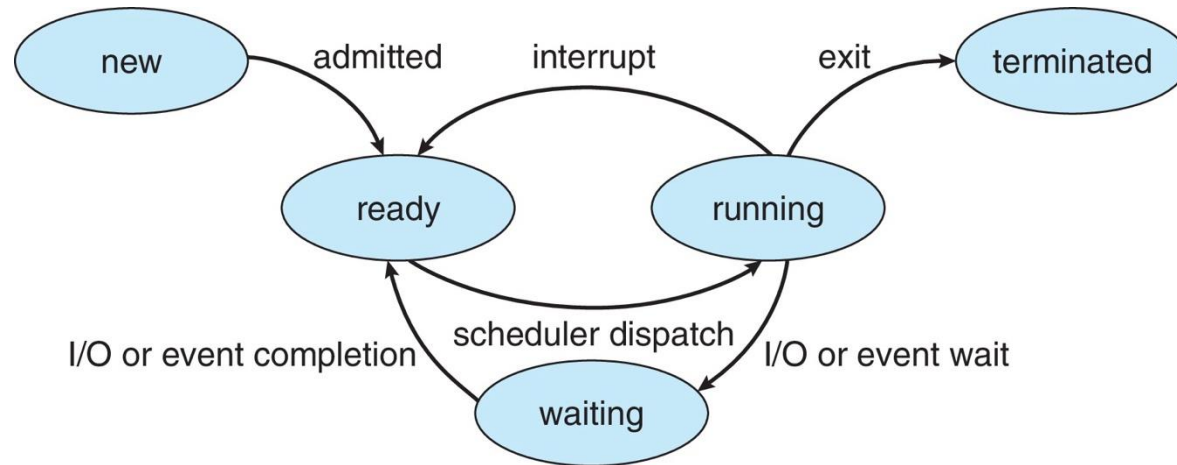
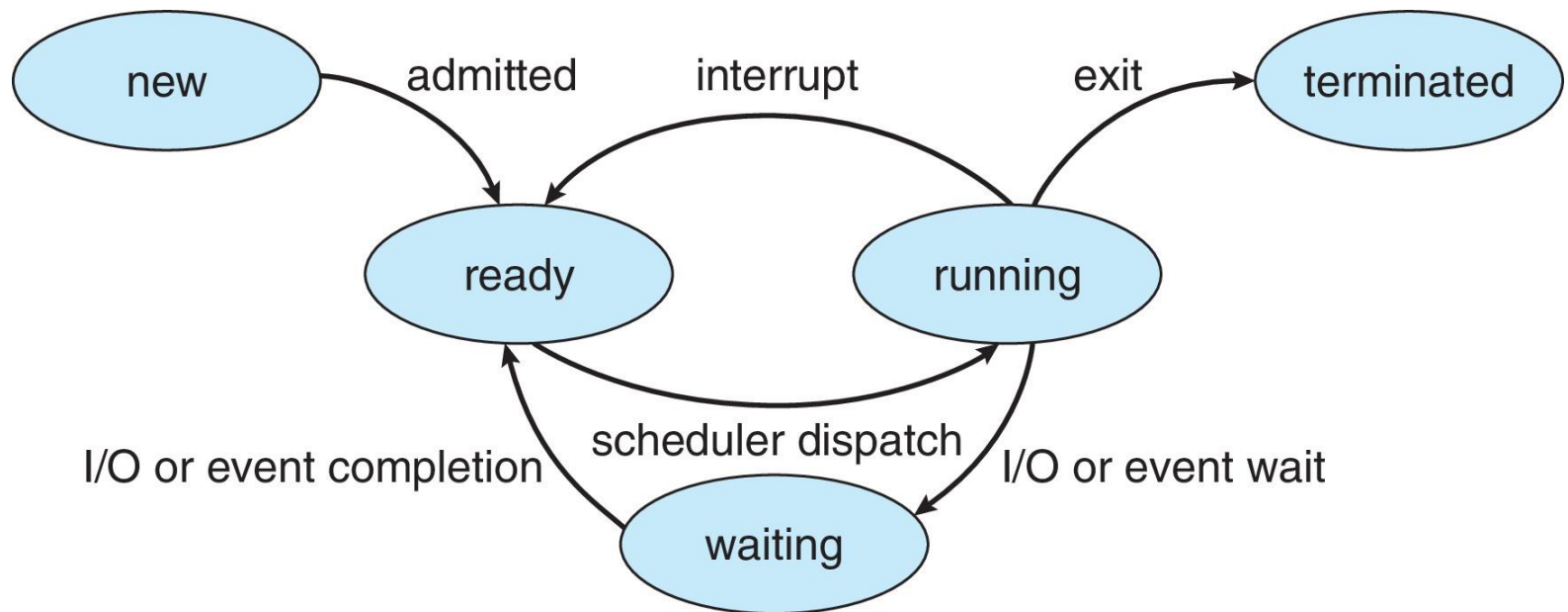




Diagram of Process State



Process State

```
128  */
129  static const char * const task_state_array[] = {
130
131      /* states in TASK_REPORT: */
132      "R (running)",          /* 0x00 */
133      "S (sleeping)",         /* 0x01 */
134      "D (disk sleep)",       /* 0x02 */
135      "T (stopped)",          /* 0x04 */
136      "t (tracing stop)",     /* 0x08 */
137      "X (dead)",             /* 0x10 */
138      "Z (zombie)",           /* 0x20 */
139      "P (parked)",           /* 0x40 */
140
141      /* states beyond TASK_REPORT: */
142      "I (idle)",              /* 0x80 */
143  };
144
```

From:

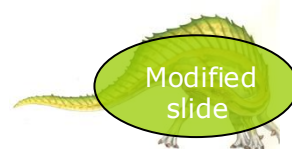
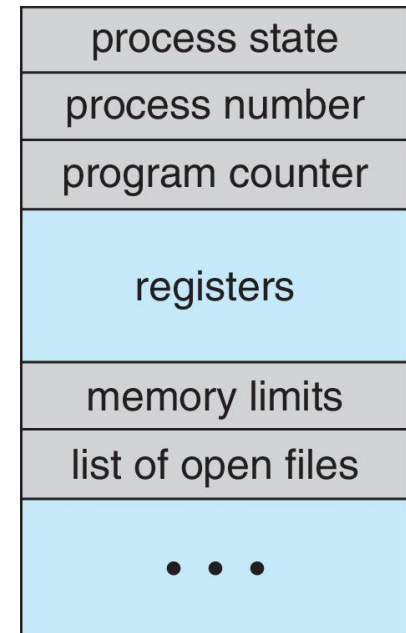
<https://github.com/torvalds/linux/blob/3950e975431bc914f7e81b8f2a2dbdf2064acb0f/fs/proc/array.c#L129-L143>



Process Control Block (PCB)

Information associated with each process (also called **task control block**)

- Process state – running, waiting, etc.
- Program counter – location of instruction to next execute
- CPU registers – contents of all process-centric registers
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files
-





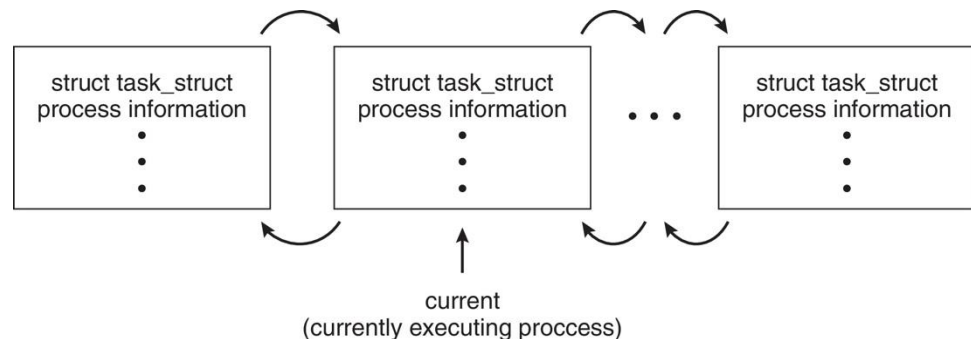
Process Representation in Linux

(in part) Represented by the C structure `task_struct`

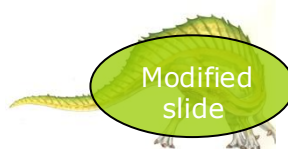
```
pid t_pid; /* process identifier */
long state; /* state of the process */
...unsigned int time_slice /* scheduling information */
struct task_struct *parent; /* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files; /* list of open files */
struct mm_struct *mm; /* address space of this
process */
```

```
/* states in TASK_REPORT: */
"R (running)", /* 0x00 */
"S (sleeping)", /* 0x01 */
"D (disk sleep)", /* 0x02 */
"T (stopped)", /* 0x04 */
"t (tracing stop)", /* 0x08 */
"X (dead)", /* 0x10 */
"Z (zombie)", /* 0x20 */
"P (parked)", /* 0x40 */

/* states beyond TASK_REPORT: */
"I (idle)", /* 0x80 */
```



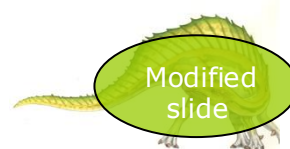
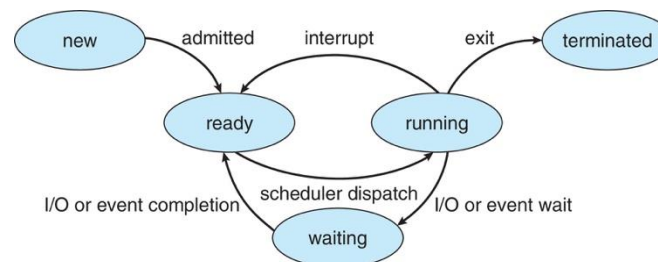
<https://github.com/torvalds/linux/blob/master/include/linux/sched.h>





Process Scheduling

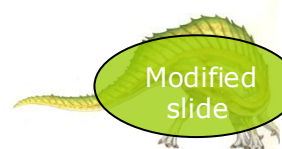
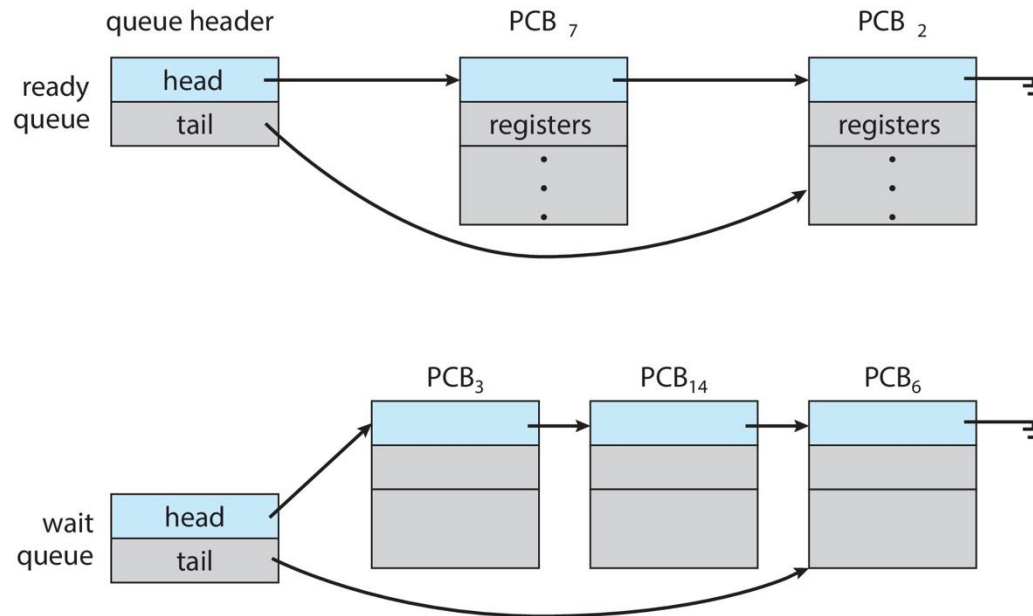
- **Process scheduler** selects among available processes for next execution on CPU core
- Goal -- Maximise CPU use, quickly switch processes onto CPU core
- Needs to maintain **scheduling queues** of processes
 - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
 - **Wait queues** – set of processes waiting for an event e.g., I/O
 - Processes migrate among the various queues





Ready and Wait Queues

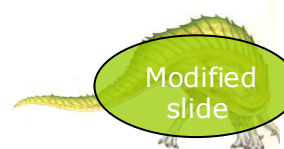
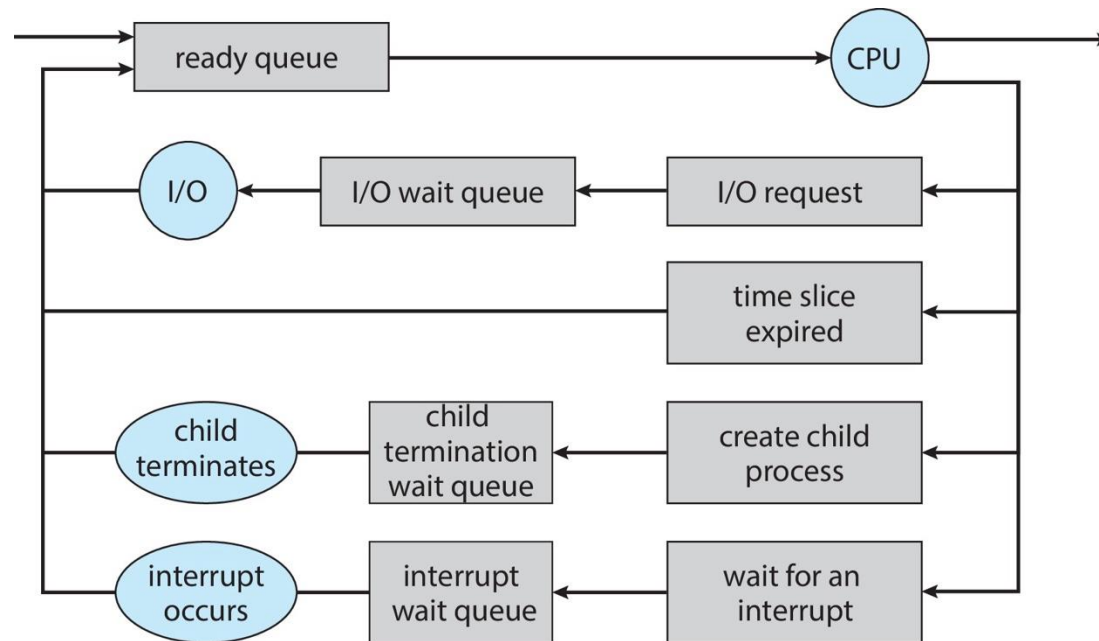
** this diagram oversimplifies:
> collapses many per-CPU requeues
> there are multiple wait queues (per resource)





Representation of Process Scheduling

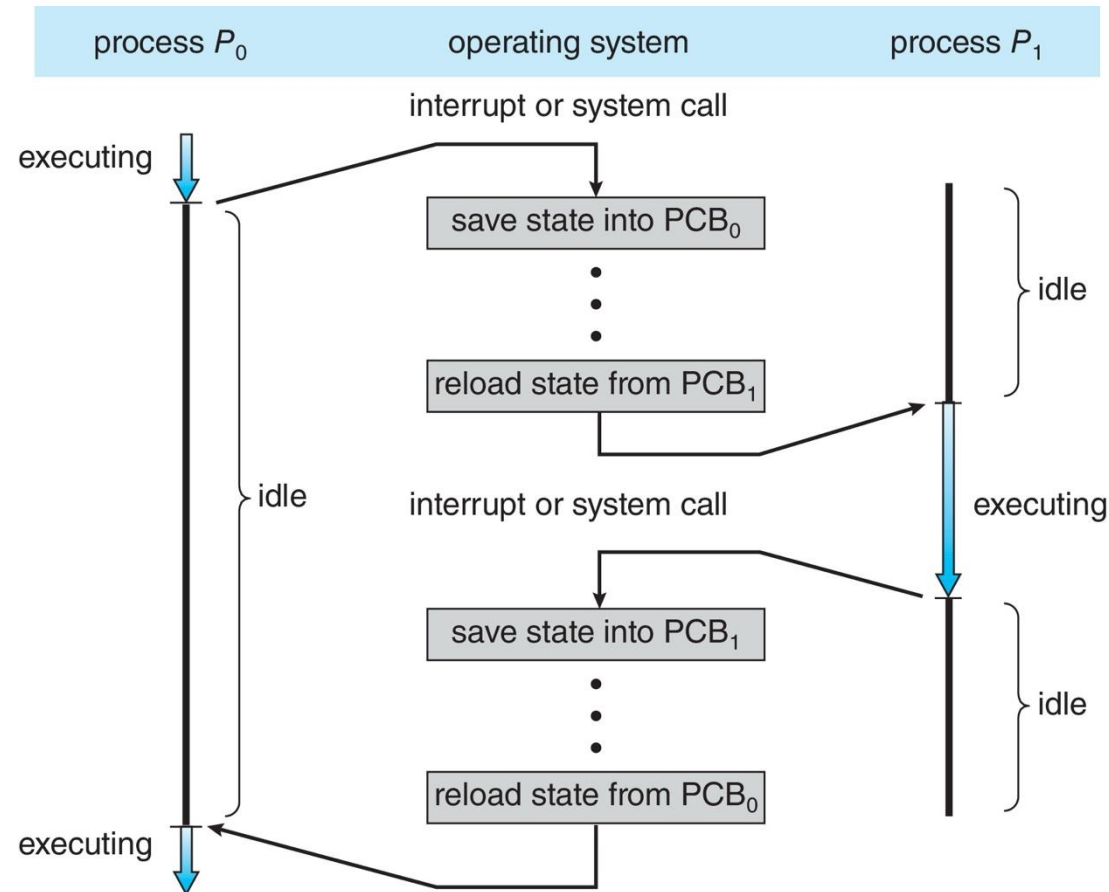
Conceptual model





CPU Switch From Process to Process

A **context switch** occurs when the CPU switches from one process to another.



process state
process number
program counter
registers
memory limits
list of open files
...





Context Switch

- **Context switch:**
 - When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process
- And the **Context** of a process represented in the PCB
- Context-switch time is **pure overhead**; **the system does no useful work while switching**
 - The more complex the OS and the PCB → the longer the context switch

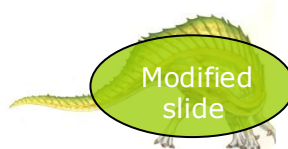
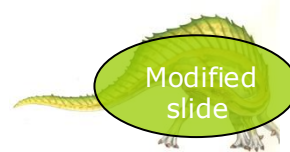
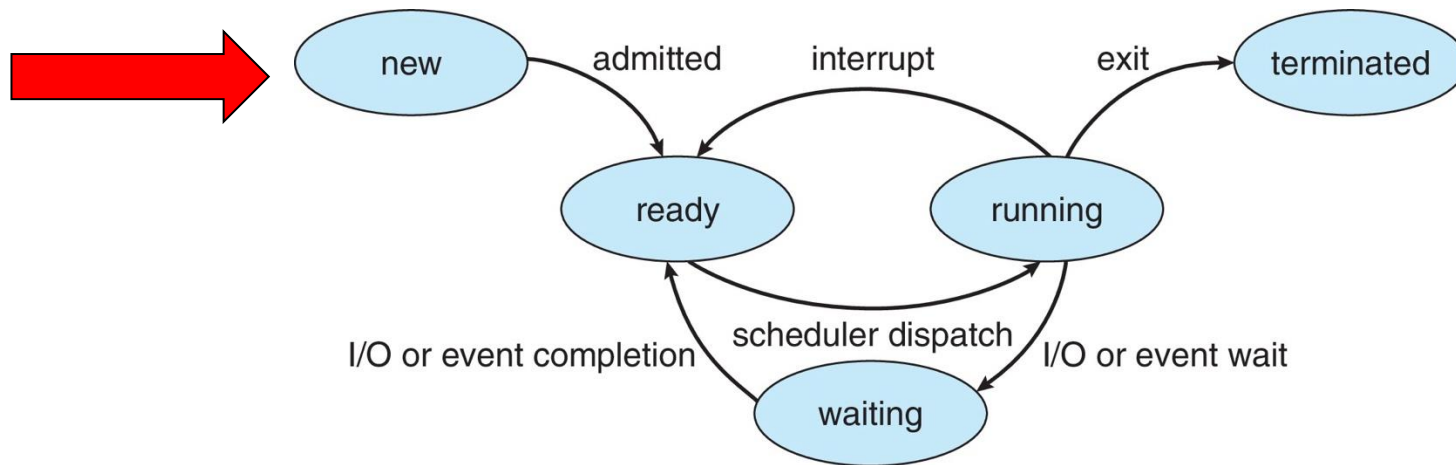




Diagram of Process State

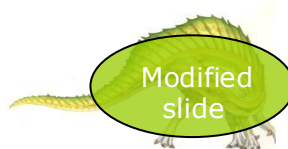




Process Creation

- When we create a new process, some choices might need to be made...
- **Resource sharing options**
 - Should... Parent and children share all resources?
 - Children share subset of parent's resources?
 - Parent and child share no resources?
- **Execution options**
 - Parent and children execute concurrently?
 - Parent waits until children terminate?

We'll come back to this a bit later...





Process Creation

- **Parent** process(es) create **children** processes, which, in turn create other processes, forming a **tree** of processes

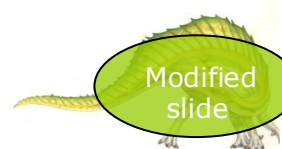
Mem Swap: 32436.5 total, 32436.5 free, 0.0 used. 26332.7 avail Mem

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
26849	ham	20	0	16976	9784	6152	R	100.0	0.0	4:16.03	python
1225	git	20	0	998060	522376	18156	S	1.7	1.6	104:53.14	bundle
1222	git	20	0	1776600	21072	11068	S	0.3	0.1	3:11.02	gitlab-workhors

Run "pstree -p"
on linux to see
process tree

```
(base) ham@hammerGPU:~$ python
Python 3.8.3 (default, Jul 2 2020, 16:21:59)
[GCC 7.3.0] :: Anaconda, Inc. on linux
Type "help", "copyright", "credits" or "license"
>>> while 1:
...     pass
... 
```

Process identified and managed via a **process identifier (pid)**



A little detour....

Processes and trees of processes

- The bash shell is a program, so it has a PID (process identified)
- We can find the PID of the current bash instance we are using by typing “**echo \$\$**” at the prompt

```
(base) ham@hammerGPU:~$ echo $$  
17882  
(base) ham@hammerGPU:~$ ps  
  PID TTY          TIME CMD  
 3101 pts/0        00:00:00 ps  
17882 pts/0        00:00:00 bash  
(base) ham@hammerGPU:~$
```

Processes and trees of processes

- Other processes?

ps

```
(base) ham@hammerGPU:~$ ps
  PID TTY          TIME CMD
  4392 pts/0        00:00:00 ps
 17882 pts/0        00:00:00 bash
(base) ham@hammerGPU:~$
```

- All processes?

ps -A

```
(base) ham@hammerGPU:~$ ps -A
  PID TTY          TIME CMD
    1 ?            00:01:15 systemd
    2 ?            00:00:00 kthreadd
    4 ?            00:00:00 kworker/0:0H
    6 ?            00:00:00 mm_percpu_wq
    7 ?            00:00:02 ksoftirqd/0
    8 ?            00:06:54 rcu_sched
    9 ?            00:00:00 rcu_bh
   10 ?            00:00:00 migration/0
   11 ?            00:00:01 watchdog/0
```

```
(base) ham@hammerGPU:~$ ps -A | wc -l
388
```

Processes and trees of processes

For example, if I used multiprocessing in python to start 10 processes:

```
(base) ham@hammerGPU:~$ echo $$  
17882  
(base) ham@hammerGPU:~$ python3 multi.py
```

```
ham@hammerGPU: ~/os_ca216/tst4 (ssh) #1 ham@ham  
from multiprocessing import *  
import random  
  
def do_a_loop():  
    while 1:  
        if random.randint(1,10000000) == 1:  
            break  
  
def main():  
    for i in range(1,10):  
        p = Process(target=do_a_loop, args=()).start()  
  
main()  
~
```

Processes and trees of processes

I could also look at output from “pstree -p 17882”

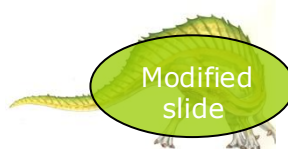
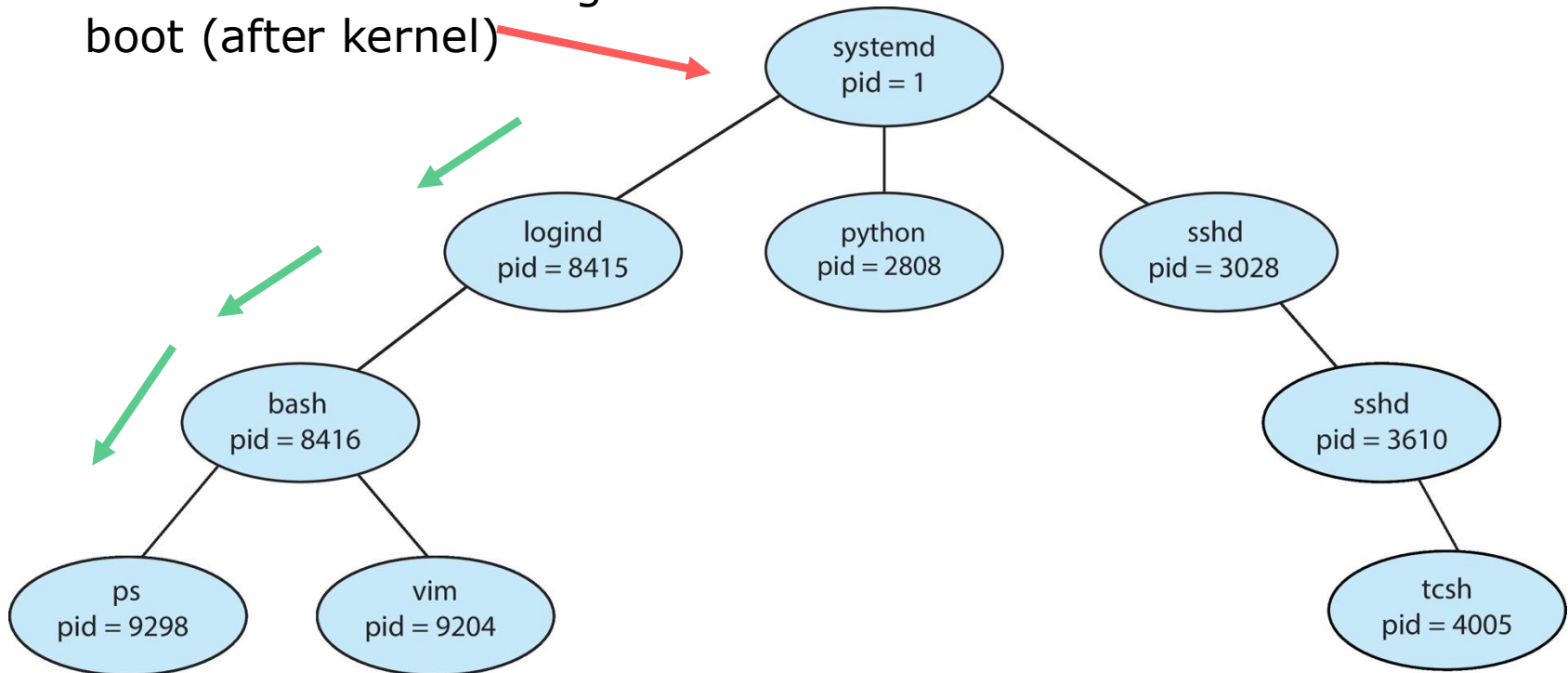
```
(base) ham@hammerGPU:~$ echo $$  
17882  
(base) ham@hammerGPU:~$ python3 multi.py
```

```
(base) ham@hammerGPU:~$ pstree -p 17882  
bash(17882)─python3(7842)─python3(7843)  
                        │python3(7844)  
                        │python3(7845)  
                        │python3(7846)  
                        │python3(7847)  
                        │python3(7848)  
                        │python3(7849)  
                        │python3(7850)  
                        │python3(7851)
```



A Tree of Processes in Linux

Systemd is the first daemon to start during boot (after kernel)





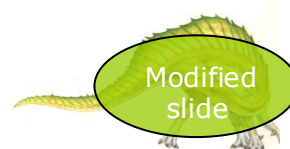
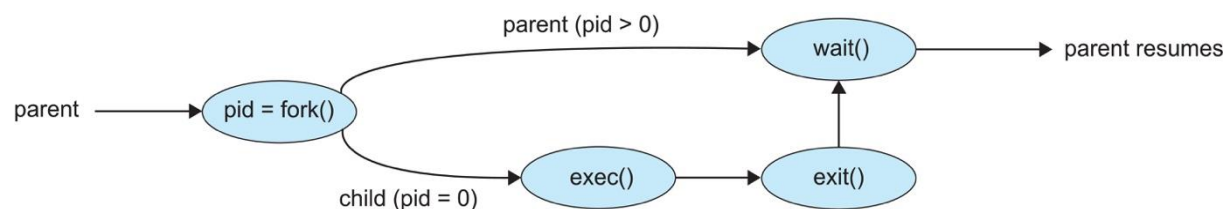
```
(base) ham@hammerGPU:~$ pstree -p 1
systemd(1)─ModemManager(7928)─{ModemManager}(7929)
          │                  └─{ModemManager}(7932)
          └─NetworkManager(8384)─{NetworkManager}(8386)
                                └─{NetworkManager}(8387)
          └─accounts-daemon(28138)─{accounts-daemon}(28139)
                                └─{accounts-daemon}(28141)
          └─acpid(32538)
          └─agetty(1861)
          └─agetty(3341)
          └─apache2(15507)─apache2(25438)
                        │   apache2(25439)
                        │   apache2(25440)
                        │   apache2(25441)
                        │   └─apache2(25442)
          └─atd(16408)
          └─avahi-daemon(7526)─avahi-daemon(7527)
          └─avahi-dnssconfd(7878)
          └─bundle(1507)─bundle(3097)─{bundle}(3099)
                        │   bundle(3103)─{bundle}(3105)
                        │   └─bundle(3106)─{bundle}(3108)
```





Process Creation

- When a process is created...
 - Child is duplicate of parent (different address space)
 - Child has a program loaded into it
- UNIX examples
 - **fork()** system call wrapper (uses **clone()**) to create new process
 - **exec()** system call wrapper used after a **fork()** to replace the process' memory space with a new program
 - Parent process calls **wait()** waiting for the child to terminate



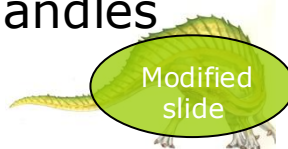


Process Creation (Cont.)

- Address space
 - Child is duplicate of parent
 - Child has a program loaded into it
- UNIX examples
 - **fork()** system call wrapper (uses **clone()**) to create new process
 - **exec()** system call wrapper used after a **fork()** to replace the process' memory space with a new program
 - Parent process calls **wait()** waiting for the child to terminate

fork() does a 'copy' of heap, stack, etc
... but the copy is not realized (because it's not used).

A mechanism called copy on write & paging (covered later) handles this.





C Program Forking Separate Process

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

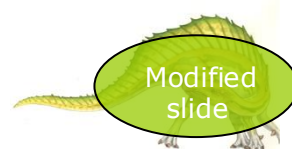
int main()
{
    pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
    }

    return 0;
}
```

**The parent gets
the child PID, and
the child PID gets
0**

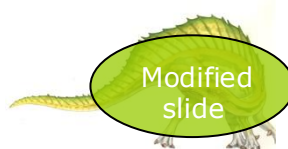




Process Termination

- Process executes last statement and then terminates by calling **exit()** (which eventually invokes the kernel).
 - Exit status is recorded by the kernel and retrieved later by the parent via **wait()**
 - Process' resources are *deallocated by operating system
- Parent may terminate the execution of children processes by sending a signal via **kill()**. Termination depends on signal type and handling. Some reasons for doing so:
 - Child has exceeded allocated resources
 - Task assigned to child is no longer required
 - The parent is exiting, and the child should not continue if its parent terminates
 - ...

*most resources are freed, however, the kernel keeps some information like PID and exit status until after the parent calls **wait()**. Between **exit()** and **wait()**, the process is a **Zombie**





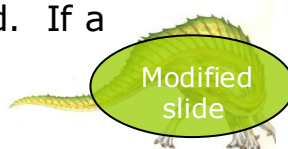
Process Termination

- On Linux/Unix (by default), if a parent terminates before its child process, the child process *continues.
- The parent process may wait for termination of a child process by using the **wait()** system call. The call returns status information and the pid of the terminated process

```
pid = wait(&status);
```

- If parent terminated without invoking **wait()**, process is an **orphan**
 - The process ends up *reparented* to PID 1 (or another reaper)
- Sometimes a process has finished running and called **exit()** but it still has a (partial) PCB .. It's a **zombie** (because the parent has not called **wait()**)
 - The zombie is cleaned up when its parent (or reaper) calls **wait()**

*Some operating systems configs do not allow child to exists if its parent has terminated. If a process terminates, then all its children must also be terminated. It uses **cascading termination where** children, grandchildren, etc., are terminated.



If I kill a process which process is the new parent?

If I kill a process which process is the new parent?

```
(base) ham@hammerGPU:~$ pstree -p 30019
bash(30019)---python(17424)---python(17425)
                             python(17426)
```

Python program that created 2 processes...

We use kill to signal the parent process

```
ham@hammerGPU:~$ kill 17424
```

PID	USER	S	%CPU	%MEM	COMMAND
17425	ham	R	100.0	0.0	python
17426	ham	R	100.0	0.0	python
1249	gitlab-ss	S	5.0	0.8	postgres

Python program that created 2 processes...

Parent is now PID=1

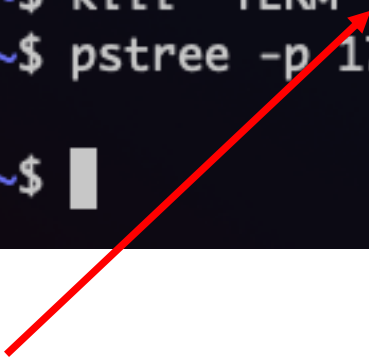
```
(base) ham@hammerGPU:~$ ps -o ppid= -p 17425
1
```

We can also kill a process group

```
(base) ham@hammerGPU:~$ pstree -p 17882
bash(17882)---python3(21478)---python3(21479)
                                   python3(21480)
```

**Python program that
created 2 processes...**

```
(base) ham@hammerGPU:~$ kill -TERM -21478
(base) ham@hammerGPU:~$ pstree -p 17882
bash(17882)
(base) ham@hammerGPU:~$
```



Using kill -TERM -PID
(notice the minus/negation)

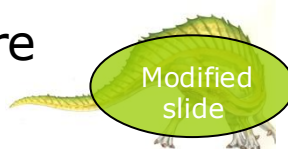




Interprocess Communication

- Processes within a system may be *independent* or *cooperating*
- **Cooperating** processes can affect or be affected by other processes (and can even share data)
- Reasons for cooperating processes:
 - Information sharing
 - Computation speedup
 - Modularity
 - ...
- Cooperating processes need **interprocess communication (IPC)**
- Two (primary) models of IPC
 - **Shared memory (we'll look at this in C)**
 - **Message passing (we'll talk about this)**

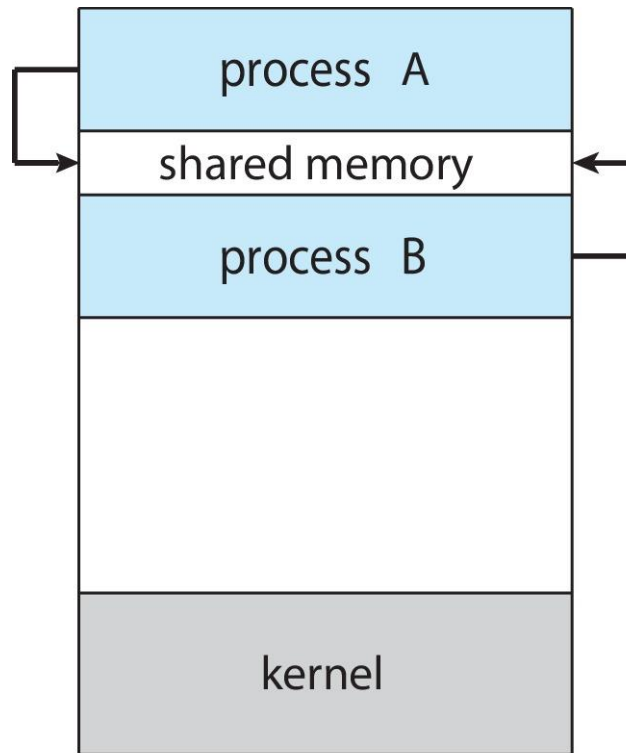
Remember: By default, processes are isolated and do not share memory.





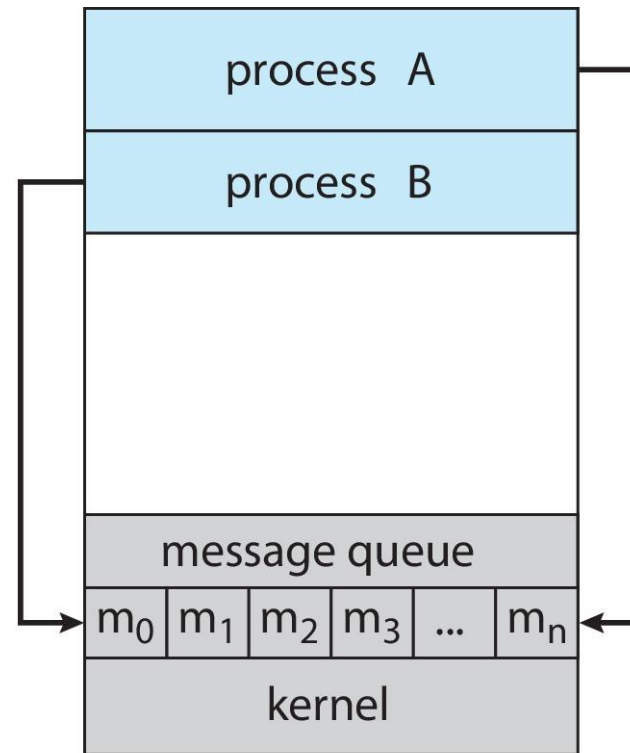
Communications Models

(a) Shared memory.



(a)

(b) Message passing.



(b)

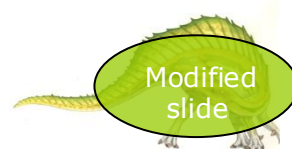




Producer-Consumer Problem

- (way to think about) Paradigm for cooperating processes:
 - *producer* process **produces** information that is **consumed** by a *consumer* process (and there's a buffer they share)
- Two variations:
 - **unbounded-buffer** places no practical limit on the size of the buffer:
 - ▶ Producer never waits
 - ▶ Consumer waits if there is no buffer to consume
 - ➔ • **bounded-buffer** assumes that there is a fixed buffer size
 - ▶ Producer must wait if all buffers are full
 - ▶ Consumer waits if there is no buffer to consume

We'll use the bounded-buffer producer-consumer problem (as a context) to explain these different methods for **IPC**





IPC – Shared Memory

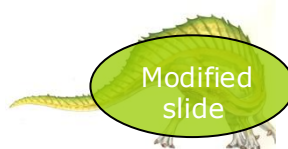
Solution 1 to the bounded-buffer problem using Shared Memory for IPC

- An area of memory shared among the processes that wish to communicate
- The communication is under the control of the user processes not the operating system.
- Major issues is to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory.



- **Synchronization** is discussed in greater detail later.

**Mutex, lock,
semaphore, ...**





Bounded-Buffer – Shared-Memory Solution

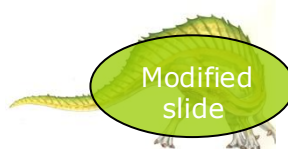
- Shared data

```
#define BUFFER_SIZE 10  
typedef struct {  
    . . .  
} item;
```

```
item buffer[BUFFER_SIZE]; ←
```

```
int in = 0; ←
```

```
int out = 0; ←
```



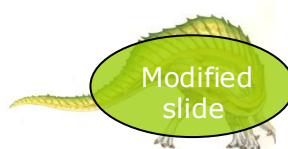


Consumer Process – Shared Memory

```
item next_consumed;

while (true) {
    while (out == in) ←
        ; /* do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;

    /* consume the item in next consumed */
}
```

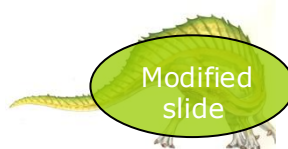
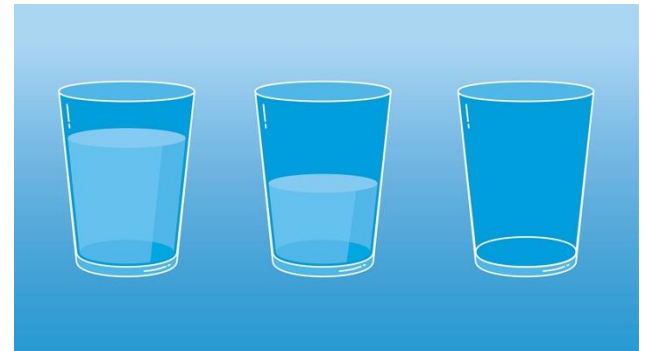




Producer Process – Shared Memory

```
item next_produced;

while (true) {
    /* produce an item in next produced */
    while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
```

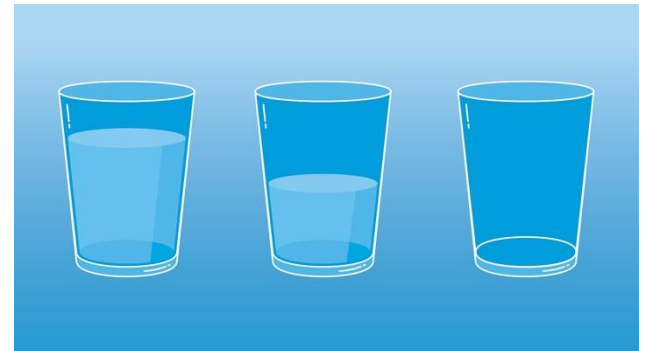




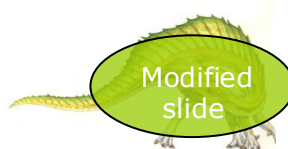
Producer Process – Shared Memory

```
item next_produced;

while (true) {
    /* produce an item in next produced */
    while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
```



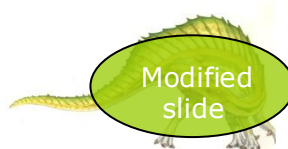
Solution is correct, but can only use **`BUFFER_SIZE-1`** elements





What about Filling all the Buffers?

- Suppose that we wanted to provide a solution to the consumer-producer problem that fills all the buffers ...
- We can do so by having an integer **counter** that keeps track of the number of full buffers.
- Initially, **counter** is set to 0.
- The integer **counter** is incremented by the producer after it produces a new buffer.
- The integer **counter** is and is decremented by the consumer after it consumes a buffer.





Consumer

```
while (true) {  
    while (counter == 0)  
        ; /* do nothing */  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
    /* consume the item in next consumed */  
}
```





Producer

```
while (true) {  
    /* produce an item in next produced */  
  
    while (counter == BUFFER_SIZE)  
        ; /* do nothing */  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
}
```





Issues with shared memory?

- **counter++** could be implemented as

```
register1 = counter
register1 = register1 + 1
counter = register1
```

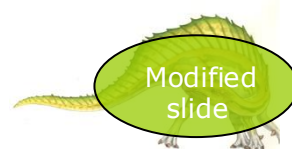
- **counter--** could be implemented as

```
register2 = counter
register2 = register2 - 1
counter = register2
```

This is called a **race condition**
- we look at situations like these and solutions later in the course ...

- Consider this execution interleaving with “count = 5” initially:

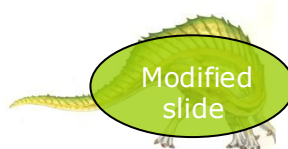
S0: producer execute	<code>register1 = counter</code>	{register1 = 5}
S1: producer execute	<code>register1 = register1 + 1</code>	{register1 = 6}
S2: consumer execute	<code>register2 = counter</code>	{register2 = 5}
S3: consumer execute	<code>register2 = register2 - 1</code>	{register2 = 4}
S4: producer execute	<code>counter = register1</code>	{counter = 6}
S5: consumer execute	<code>counter = register2</code>	{counter = 4}





Examples of IPC Systems - POSIX

- Let's look at setting up shared memory on Linux
 - In Linux, this is a file, but we map it into memory!
- POSIX Shared Memory
 - Process first creates shared memory segment
`shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);`
 - Set the size of the object
`ftruncate(shm_fd, 4096);`
 - Use `mmap()` to memory-map a file pointer to the shared memory object
 - Reading and writing to shared memory is done by using the pointer returned by `mmap()`.





Shared memory example

./producer

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <sys/types.h>
```

```
int main()
{
    const int SIZE = 4096;
    const char *name = "OS";
    const char *message0= "Studying ";
    const char *message1= "Operating Systems ";
    const char *message2= "Is Fun!";

    int shm_fd;
    void *ptr;

    /* create the shared memory segment */
    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);

    /* configure the size of the shared memory segment */
    ftruncate(shm_fd,SIZE);

    /* now map the shared memory segment in the address space of the pr
    ptr = mmap(0,SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
    if (ptr == MAP_FAILED) {
        printf("Map failed\n");
        return -1;
    }

    /**
     * Now write to the shared memory region.
     *
     * Note we must increment the value of ptr after each write.
     */
    sprintf(ptr,"%s",message0);
    ptr += strlen(message0);
    sprintf(ptr,"%s",message1);
    ptr += strlen(message1);
    sprintf(ptr,"%s",message2);
    ptr += strlen(message2);

    return 0;
}
```



Shared memory example

./consumer

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>
```

```
int main()
{
    const char *name = "OS";
    const int SIZE = 4096;

    int shm_fd;
    void *ptr;
    int i;

    /* open the shared memory segment */
    shm_fd = shm_open(name, O_RDONLY, 0666);
    if (shm_fd == -1) {
        printf("shared memory failed\n");
        exit(-1);
    }

    /* now map the shared memory segment in the address space of the process */
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);
    if (ptr == MAP_FAILED) {
        printf("Map failed\n");
        exit(-1);
    }

    /* now read from the shared memory region */
    printf("%s", (char *)ptr);

    /* remove the shared memory segment */
    if (shm_unlink(name) == -1) {
        printf("Error removing %s\n", name);
        exit(-1);
    }

    return 0;
}
```





Shared memory example

```
(base) ham@hammerGPU:~/os_ca216/shm$ ls
shm-posix-consumer.c  shm-posix-producer.c
(base) ham@hammerGPU:~/os_ca216/shm$ gcc shm-posix-consumer.c -o consumer -lrt
(base) ham@hammerGPU:~/os_ca216/shm$ gcc shm-posix-producer.c -o producer -lrt
(base) ham@hammerGPU:~/os_ca216/shm$ ./producer
(base) ham@hammerGPU:~/os_ca216/shm$ ./consumer
This is CA216 Operating Systems
(base) ham@hammerGPU:~/os_ca216/shm$
```

Note:

1. You need to use `-lrt` with `gcc` (so it knows to use `rt` library)
2. That the shared memory persisted, even after producer exited!
3. Consumer could have have been run as a different user ...

```
ham2@hammerGPU:~$ /tmp/consumer
This is CA216 Operating Systems
Error removing OS
```

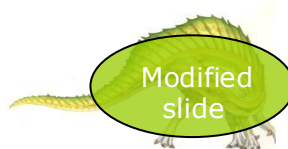




IPC – Message Passing

Solution 2 to the bounded-buffer problem using IPC & Message Passing

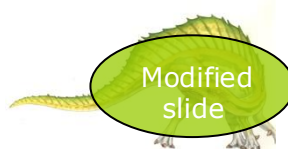
- Processes can communicate with each other without resorting to shared variables
- IPC (Message Passing) facility provides two operations:
 - `send(message)`
 - `receive(message)`
- The *message* size is either fixed or variable





Message Passing (Cont.)

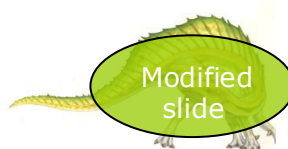
- If processes ***P*** and ***Q*** wish to communicate, they need to:
 - Establish a ***communication link*** between them
 - Exchange messages via **send / receive**
- (some) Implementation considerations:
 - Can a link be associated with more than **two processes**?
 - How many links can there be **between every pair** of communicating processes?
 - What is the capacity of a link? (buffer size? queue length?)
 - Will the size of a message that the link can accommodate be fixed or variable?
 - Will a link be **unidirectional** or **bi-directional**?





Direct / Indirect Communication

- **Direct** – The sender names the receiver explicitly:
 - **send** (P , *message*) – send a message to process P
 - **receive**(Q , *message*) – receive a message from process Q
- **Indirect** - Messages are directed and received from **mailboxes** (also referred to as ports,....) - basically some intermediary entity:
 - Each **mailbox** has a unique id
 - Processes can communicate only if they share a mailbox





Synchronization

Message passing may be either blocking or non-blocking

- **Blocking** is considered **synchronous**
 - **Blocking send()** -- the sender is blocked until the message is received
 - **Blocking receive()** -- the receiver is blocked until a message is available
- **Non-blocking** is considered **asynchronous**
 - **Non-blocking send** -- the sender sends the message and continue
 - **Non-blocking receive** -- the receiver receives:
 - ▶ A valid message, or
 - ▶ Null message
- Different combinations possible
 - If both send and receive block, we have a **rendezvous**





Producer-Consumer: Message Passing

- Producer

```
message next_produced;  
while (true) {  
    /* produce an item in next_produced */  
  
    send(next_produced) ;  
}
```

- Consumer

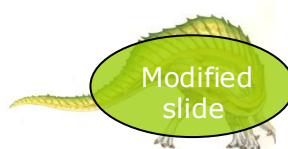
```
message next_consumed;  
while (true) {  
    receive(next_consumed)  
  
    /* consume the item in next_consumed */  
}
```





(considerations) Buffering

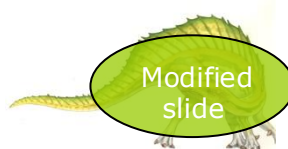
- Queue of messages attached to the link.
- Implemented in one of three ways
 1. Zero capacity – no messages are queued on a link.
Sender must wait for receiver (rendezvous)
 2. Bounded capacity – finite length of n messages
Sender must wait if link full
 3. Unbounded capacity – infinite length
Sender never waits (but practically, what about memory RAM issues?)





Pipes

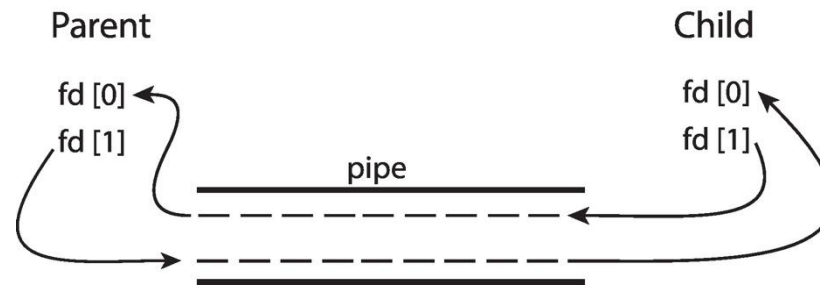
- Acts as a **conduit** allowing processes to communicate
- Configuration:
 - (typically) Communication is **unidirectional**
 - FIFO
 - ...
- **Ordinary pipes** – Ordinary pipes are unnamed and accessible only to processes that inherit the pipe's file descriptors (typically parent and child).
- **Named pipes** – can be accessed without a parent-child relationship.





Ordinary Pipes

- Ordinary Pipes allow communication in standard producer-consumer style
- Producer writes to one end (the **write-end** of the pipe)
- Consumer reads from the other end (the **read-end** of the pipe)
- Require parent-child relationship between communicating processes



- Windows calls these **anonymous pipes**



Pipe example (ordinary)

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <string.h>

#define BUFFER_SIZE 25
#define READ_END    0
#define WRITE_END   1
```

```
* @author Gagne, Galvin, Silberschatz
* Operating System Concepts – Tenth Edition
* Copyright John Wiley & Sons – 2018
```

```
int main(void)
{
    char write_msg[BUFFER_SIZE] = "Greetings";
    char read_msg[BUFFER_SIZE];
    pid_t pid;
    int fd[2];

    /* create the pipe */
    if (pipe(fd) == -1) {
        fprintf(stderr, "Pipe failed");
        return 1;
    }

    /* now fork a child process */
    pid = fork();

    if (pid < 0) {
        fprintf(stderr, "Fork failed");
        return 1;
    }

    if (pid > 0) { /* parent process */
        /* close the unused end of the pipe */
        close(fd[READ_END]);

        /* write to the pipe */
        write(fd[WRITE_END], write_msg, strlen(write_msg)+1);

        /* close the write end of the pipe */
        close(fd[WRITE_END]);
    }
    else { /* child process */
        /* close the unused end of the pipe */
        close(fd[WRITE_END]);

        /* read from the pipe */
        read(fd[READ_END], read_msg, BUFFER_SIZE);
        printf("child read %s\n", read_msg);

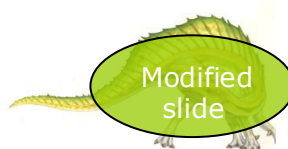
        /* close the write end of the pipe */
        close(fd[READ_END]);
    }

    return 0;
}
```




Named Pipes

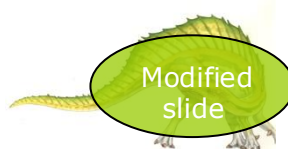
- Named Pipes are more flexible in how processes find each other
- Communication can be bidirectional
- No parent-child relationship is necessary between the communicating processes
- **mkfifo**
example
- Several processes can use the named pipe for communication
- Provided on both UNIX and Windows systems
- Appears as a file, processes attach to it for IPC





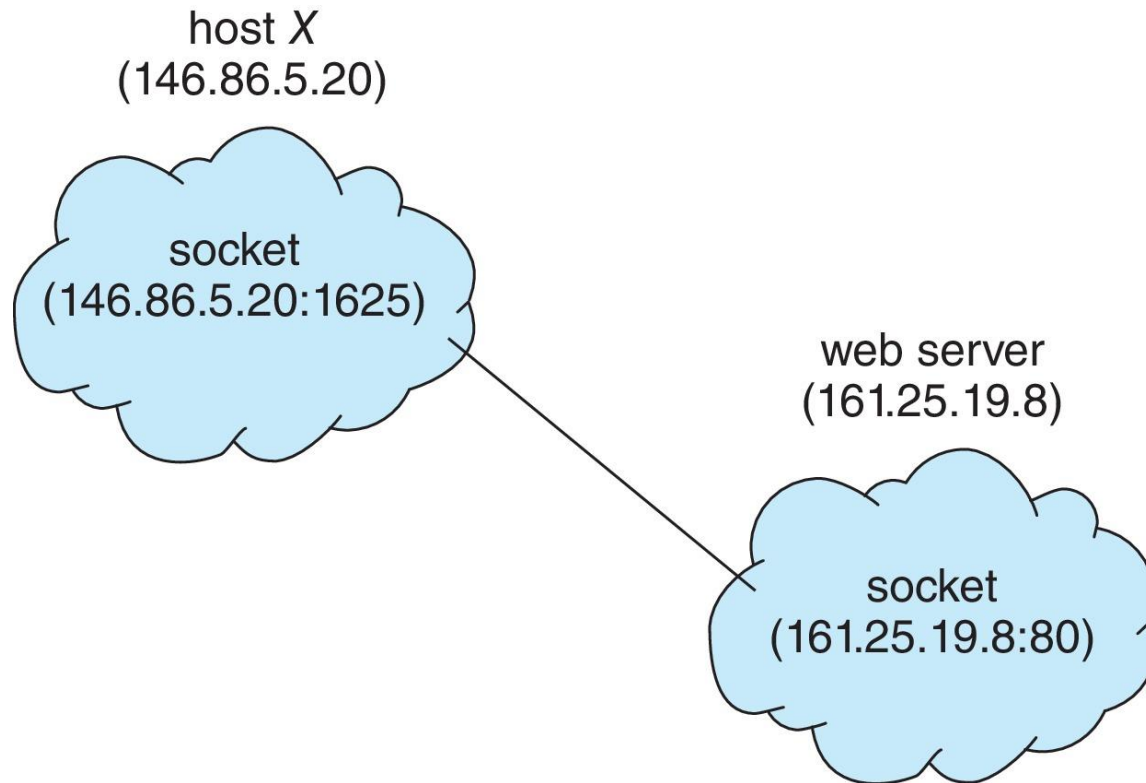
Communications in Client-Server Systems (Sockets)

- A **socket** is defined as an endpoint for communication
- Concatenation of IP address and **port** – a number included at start of message packet to differentiate network services on a host
- The socket **161.25.19.8:1625** refers to port **1625** on host **161.25.19.8**
- Communication consists between a pair of sockets
- All ports below 1024 are **well known**, used for standard services
- Special IP address 127.0.0.1 (**loopback**) to refer to system on which process is running





Socket Communication



Recap

- Interprocess Communication (IPC)
 - Shared memory
 - Message passing
- I mentioned about race conditions



Issues with shared memory?

- `counter++` could be implemented as

```
register1 = counter
register1 = register1 + 1
counter = register1
```

This is called a **race condition**
- we look at situations like these and solutions later in the course ...

- `counter--` could be implemented as

```
register2 = counter
register2 = register2 - 1
counter = register2
```

- Consider this execution interleaving with "count = 5" initially:

S0: producer execute	<code>register1 = counter</code>	{register1 = 5}
S1: producer execute	<code>register1 = register1 + 1</code>	{register1 = 6}
S2: consumer execute	<code>register2 = counter</code>	{register2 = 5}
S3: consumer execute	<code>register2 = register2 - 1</code>	{register2 = 4}
S4: producer execute	<code>counter = register1</code>	{counter = 6}
S5: consumer execute	<code>counter = register2</code>	{counter = 4}





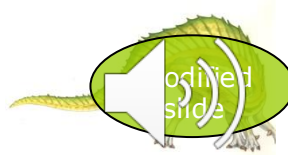
Signals & Signal Handling for processes





Signal Handling

- **Signals** are used in UNIX systems to notify a process that a particular event has occurred.
- A **signal handler** is used to process signals
 1. Signal is generated by particular event
 2. Signal is delivered to a process
 3. Signal is handled by one of two signal handlers:
 1. default
 2. user-defined
- Every signal has **default handler** defined by the kernel
 - **User-defined signal handler** can override default
 - For single-threaded, signal delivered to process
 - Some signals cannot be caught or ignored (e.g. SIGKILL, SIGSTOP)





Signal handling

- When a signal is sent to a process, it interrupts its normal execution, and either:
 - Uses the default signal handler
 - Or uses a process defined signal handler
- Use `kill -l`, to see a list of available signals

```
(base) ham@hammerGPU:~$ kill -l
1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
6) SIGABRT     7) SIGBUS      8) SIGFPE      9) SIGKILL     10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM    15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD    18) SIGCONT    19) SIGSTOP    20) SIGTSTP
21) SIGTTIN    22) SIGTTOU    23) SIGURG     24) SIGXCPU    25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF    28) SIGWINCH    29) SIGIO       30) SIGPWR
31) SIGSYS     34) SIGRTMIN   35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX
```





Signal handling

Examples:

- SIGINT = when you press ctrl + c
 - SIGQUIT = when you press ctrl + \
 - SIGTERM = tell the process to terminate (gracefully)
 - SIGFPE = illegal arithmetic error e.g. (divide by zero)
 -
-
- We send these to a process using kill.. E.g.
 - kill 123456
 - kill -SIGTERM 123456 (same as above, default is SIGTERM)

 - Or in C:
kill(pid, SIGTERM); // or any of the list of available signals





Signal handling (setup)

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void sighandler(int signum)
{
    printf("Received a signal %d\n", signum);
}

int main()
{
    signal(SIGINT, sighandler);
    while(1);
    return 0;
}
```

```
(base) ham@hammerGPU:~/tmp$ gcc sig.c -o sig
(base) ham@hammerGPU:~/tmp$ ./sig
Received a signal 2
```

I found the
PID using
'ps a'

```
(base) ham@hammerGPU:~$ kill -SIGINT 2855
```



Processes & Threads



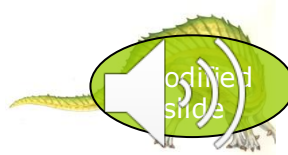


Benefits

Threads and processes are similar in that they allow concurrent/parallel execution of code (depending on OS & programming language)

- **Responsiveness** – may allow continued execution if part of process is blocked, especially important for user interfaces (**like processes**)
- **Resource Sharing** – threads share resources of process, easier than shared memory or message passing (**unlike between child/parent/child processes**)
- **Economy** – cheaper than process creation, thread switching lower overhead than context switching
- **Scalability** – effective way to take advantage of multicore architectures

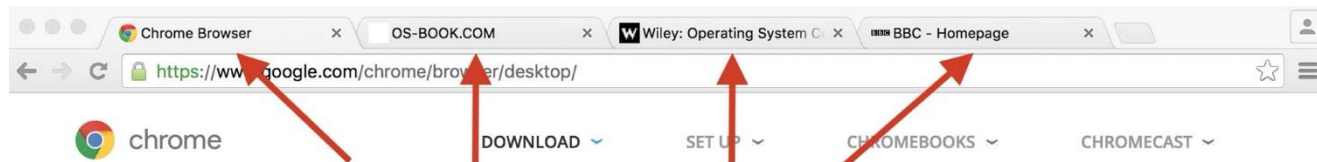
So, what's the difference between a process and a thread?



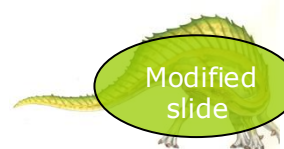


Multiprocess Architecture – Chrome Browser

- Many web browsers in the past as single process (**maybe some still do?**)
 - If one web site causes trouble, entire browser can hang or crash
- Google Chrome Browser is multiprocess with 3 different types of processes:
 - **Browser** process manages user interface, disk and network I/O
 - **Renderer** process renders web pages, deals with HTML, Javascript. A new renderer created for each website opened
 - ▶ Runs in **sandbox** restricting disk and network I/O, minimizing effect of security exploits
 - **Plug-in** process for each type of plug-in

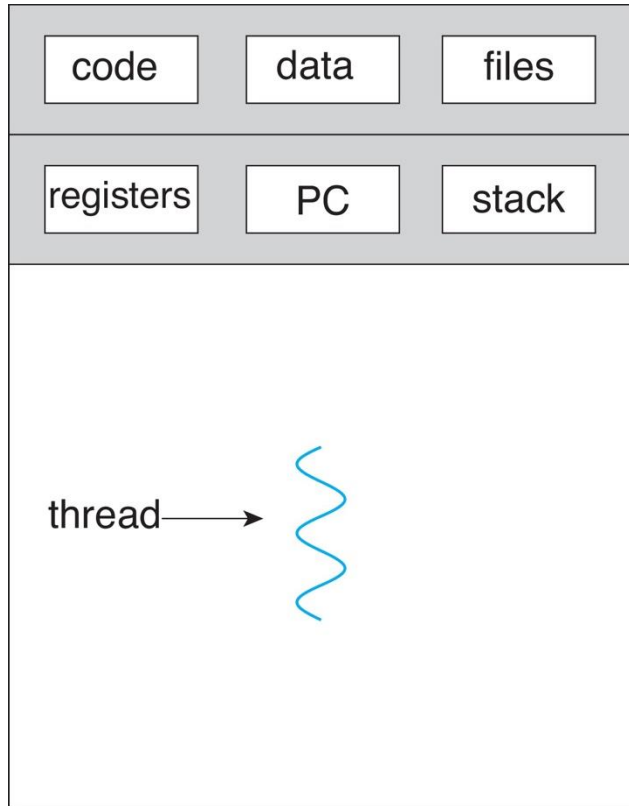


Each tab represents a separate process.

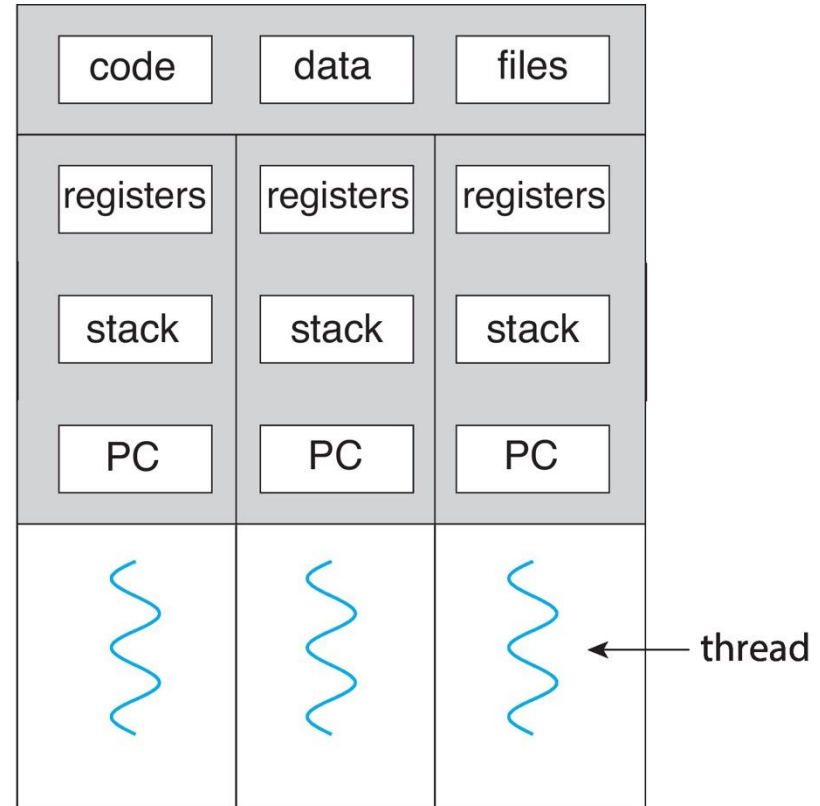




Single and Multithreaded Processes



single-threaded process



multithreaded process





Multicore Programming

- Example to show when we fork() a new process, it has it's own memory i.e. parent process does not see change to val from child, and vice versa!

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int var = 5;

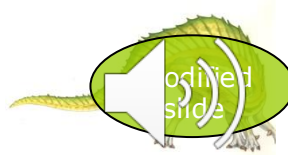
int main(int argc, char**argv)
{
    pid_t fork_ret;

    fork_ret = fork();

    if (fork_ret == 0) // in the child
    {
        var = 100;
        printf("In the child with pid %d, and the value of var is %d\n", getpid(), var);
    }
    else // in the parent
    {
        sleep(4); // sleep in the parent so child has had a chance to run for a while
        printf("In the parent with pid %d, and the value of var is %d\n", getpid(), var);
        wait(NULL);
    }

    return 0;
}
```

```
(base) ham@hammerGPU:~/ca216/week4$ ./fork_diff
In the child with pid 3264225, and the value of var is 100
In the parent with pid 3264224, and the value of var is 5
```



```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
```

```
int shared = 0; // shared memory (same address space)
```

```
void* writer(void* arg)
```

```
{
    sleep(1); // give reader time to start
    shared = 42;
    printf("Writer: set shared = %d\n", shared);
    return NULL;
}
```

```
void* reader(void* arg)
```

```
{
    printf("Reader: initial shared = %d\n", shared);

    while (shared == 0) {
        // busy wait
    }

    printf("Reader: saw shared change to %d\n", shared);
    return NULL;
}
```

```
int main()
```

```
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, reader, NULL);
    pthread_create(&t2, NULL, writer, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
}
```

```
return 0;
```

Click to add notes

programming

Example to show that in a process with two threads, they can see the same variable*

**there are caveats to allowing direct shared access like this which we discuss later e.g. race conditions*

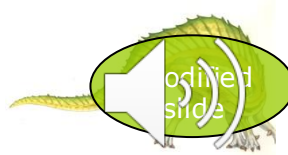
```
ham@ichbin:~/os$ gcc thread_example.c -pthread -o thread_example
ham@ichbin:~/os$ ./thread_example
Reader: initial shared = 0
Reader: saw shared change to 42
Writer: set shared = 42
```





Multicore Programming

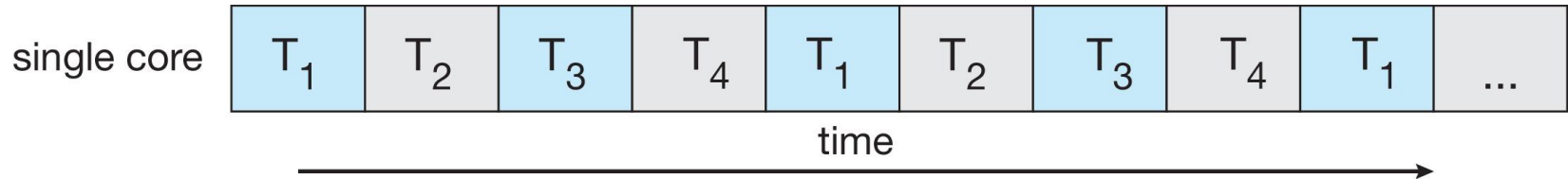
- **Multicore** or **multiprocessor** systems put pressure on programmers, challenges include:
 - **Dividing activities**
 - **Balancing**
 - **Data splitting**
 - **Data dependency**
 - **Testing and debugging (e.g. heisenbugs)**
- **Parallelism** implies a system can perform more than one task simultaneously
- **Concurrency** supports more than one task making progress
 - Single processor / core, scheduler providing concurrency



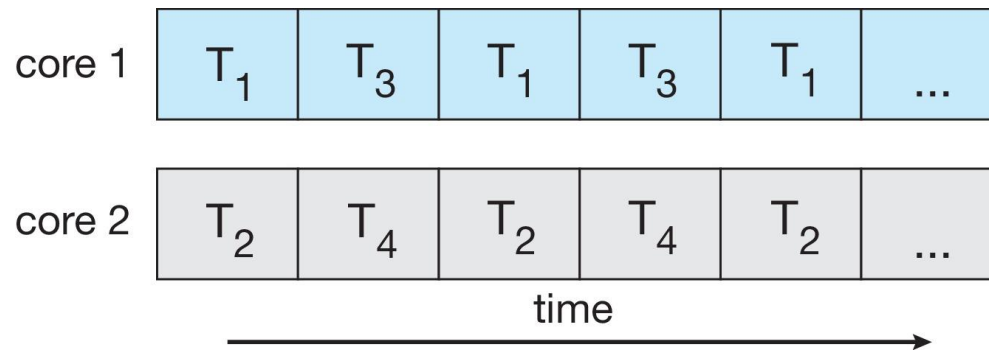


Concurrency vs. Parallelism

- **Concurrent execution on single-core system:**



- **Parallelism on a multi-core system:**





Are Python threads real threads?

```
from threading import Thread

def my_fun(c):
    while True:
        pass

for i in range(3):
    Thread(target=my_fun, args=(1000,)).start()
~
~
```

If you run 'top -H', you will see this **does not** create multiple processes .. but threads.. but they cannot run at the same time (due to GIL) – but are running concurrently.

```
from multiprocessing import Process

def my_fun(c):
    while True:
        pass

for i in range(3):
    Process(target=my_fun, args=(1000,)).start()
```

If you run 'top', you will see this **does** create multiple processes ... each using close to 100% of a cpu ... but they are separate processes not threads

Note: This is an edgecase specific to Python... In (most) other programming languages each of the threads (left) would be able to run at the same time!





```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
```

```
int sum; /* this data is shared by the thread(s) */
```

```
void *runner(void *param); /* the thread */
```

```
int main(int argc, char *argv[])
```

```
{
```

```
    pthread_t tid; /* the thread identifier */
```

```
    pthread_attr_t attr; /* set of attributes for the thread */
```

```
    if (argc != 2) {
```

```
        fprintf(stderr, "usage: a.out <integer value>\n");
```

```
        /*exit(1);*/
```

```
        return -1;
```

```
    }
```

```
    if (atoi(argv[1]) < 0) {
```

```
        fprintf(stderr, "Argument %d negative\n", atoi(argv[1]));
```

```
        /*exit(1);*/
```

```
        return -1;
```

```
    }
```

```
    /* get the default attributes */
```

```
    pthread_attr_init(&attr);
```

```
    /* create the thread */
```

```
    pthread_create(&tid, &attr, runner, argv[1]);
```

```
    /* now wait for the thread to exit */
```

```
    pthread_join(tid, NULL);
```

```
    printf("sum = %d\n", sum);
```

```
}
```

```
/**
 * A pthread program illustrating how to
 * create a simple thread and some of the pthread API
 * This program implements the summation function where
 * the summation operation is run as a separate thread.
 *
 * Most Unix/Linux/OS X users
 * gcc thrd.c -lpthread
 *
 * Figure 4.11
 *
 * @author Gagne, Galvin, Silberschatz
 * Operating System Concepts - Tenth Edition
 * Copyright John Wiley & Sons - 2018
 */
```

```
/**
```

```
 * The thread will begin control in this function
```

```
 */
```

```
void *runner(void *param)
```

```
{
```

```
    int i, upper = atoi(param);
```

```
    sum = 0;
```

```
    if (upper > 0) {
```

```
        for (i = 1; i <= upper; i++)
```

```
            sum += i;
```

```
    }
```

```
    pthread_exit(0);
```

```
}
```





- I've shared this example on the Loop page (thrd-posix.c)
 - You should examine/modify/play/compile it
 - Can you modify the code to create more than one thread?
 - What do you see if you use the 'top' command? (try shift + H)
 - Or the ps command? or with flags (i.e. ps -eLf) hint: LWP shows the thread ID
 - Compare this to case where you might only use fork() Would the child process in the case of a fork() be able to modify "int sum;"?
- We come back to pthreads later when covering mutexes and semaphores





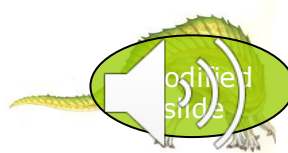
Linux Process vs Threads

- Thread creation is done through `clone()` system call
- `clone()` allows a child task to share the address space of the parent task (process)
 - Example flags control behavior



flag	meaning
<code>CLONE_FS</code>	File-system information is shared.
<code>CLONE_VM</code>	The same memory space is shared.
<code>CLONE_SIGHAND</code>	Signal handlers are shared.
<code>CLONE_FILES</code>	The set of open files is shared.

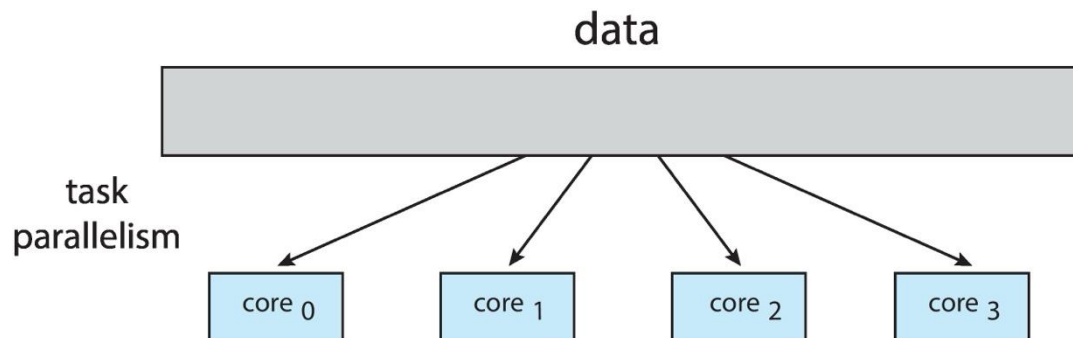
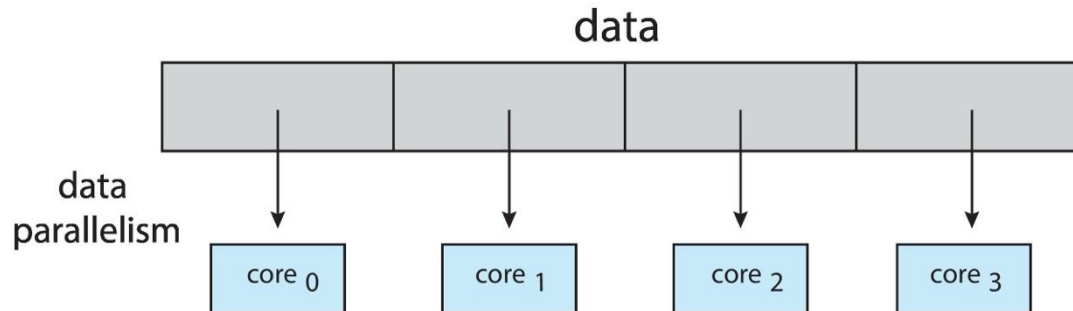
Both `fork()` and `pthread_create()` use the clone system call...



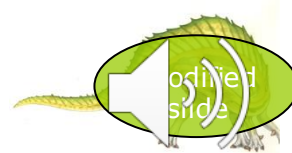


Multicore Programming

- Types of parallelism
 - **Data parallelism** – distributes subsets of the same data across multiple cores, **same operation on each**
 - **Task parallelism** – distributing threads across cores, each thread performing **unique operation**



Example:
Spellchecker
+ keyword
checker
+ ...





Amdahl's Law

- Identifies performance gains from adding additional cores to an application that has both serial and parallel components
- S is serial portion
- N processing cores

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- That is, if application is 75% parallel / 25% serial, moving from 1 to 2 cores results in speedup of 1.6 times
- As N approaches infinity, speedup approaches $1 / S$

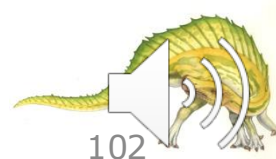
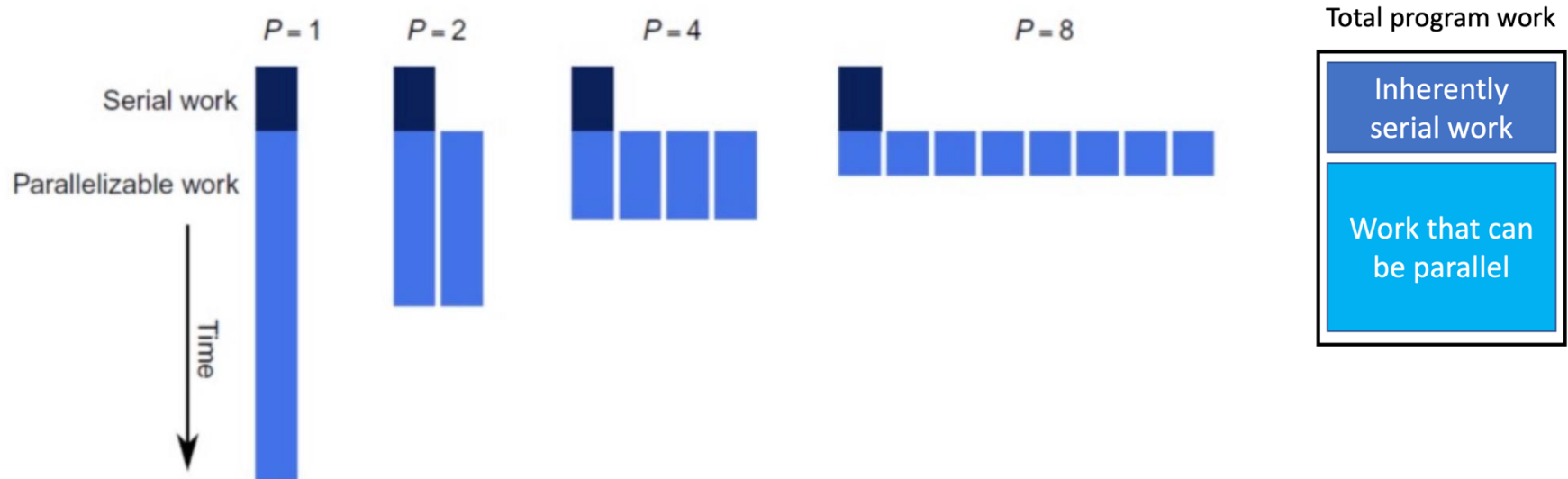
Serial portion of an application impacts performance gained by adding additional cores





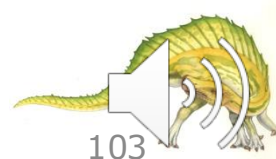
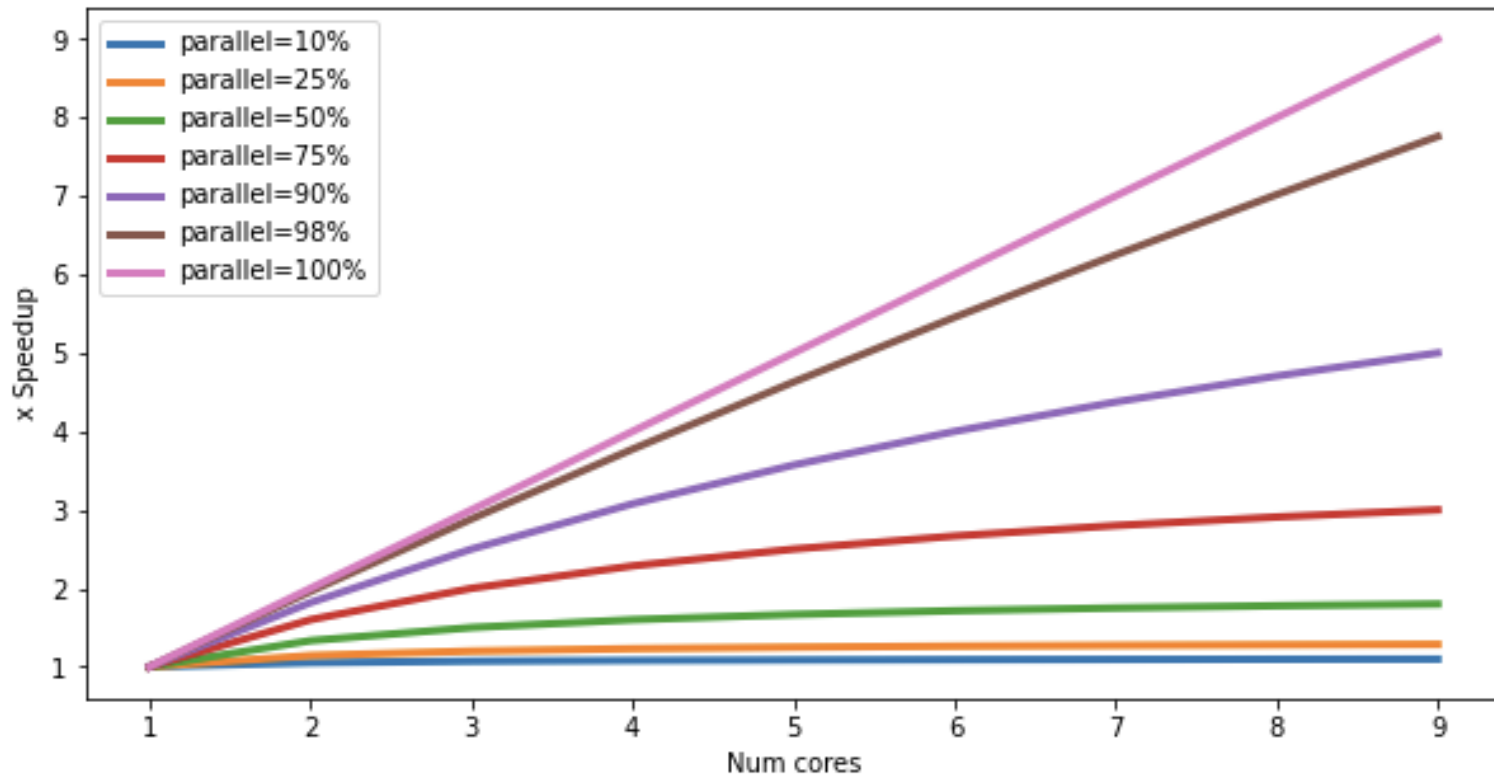
Amdahl's Law

This can be better seen in the following schematic:



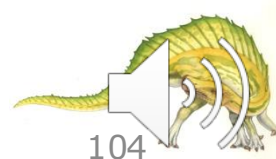
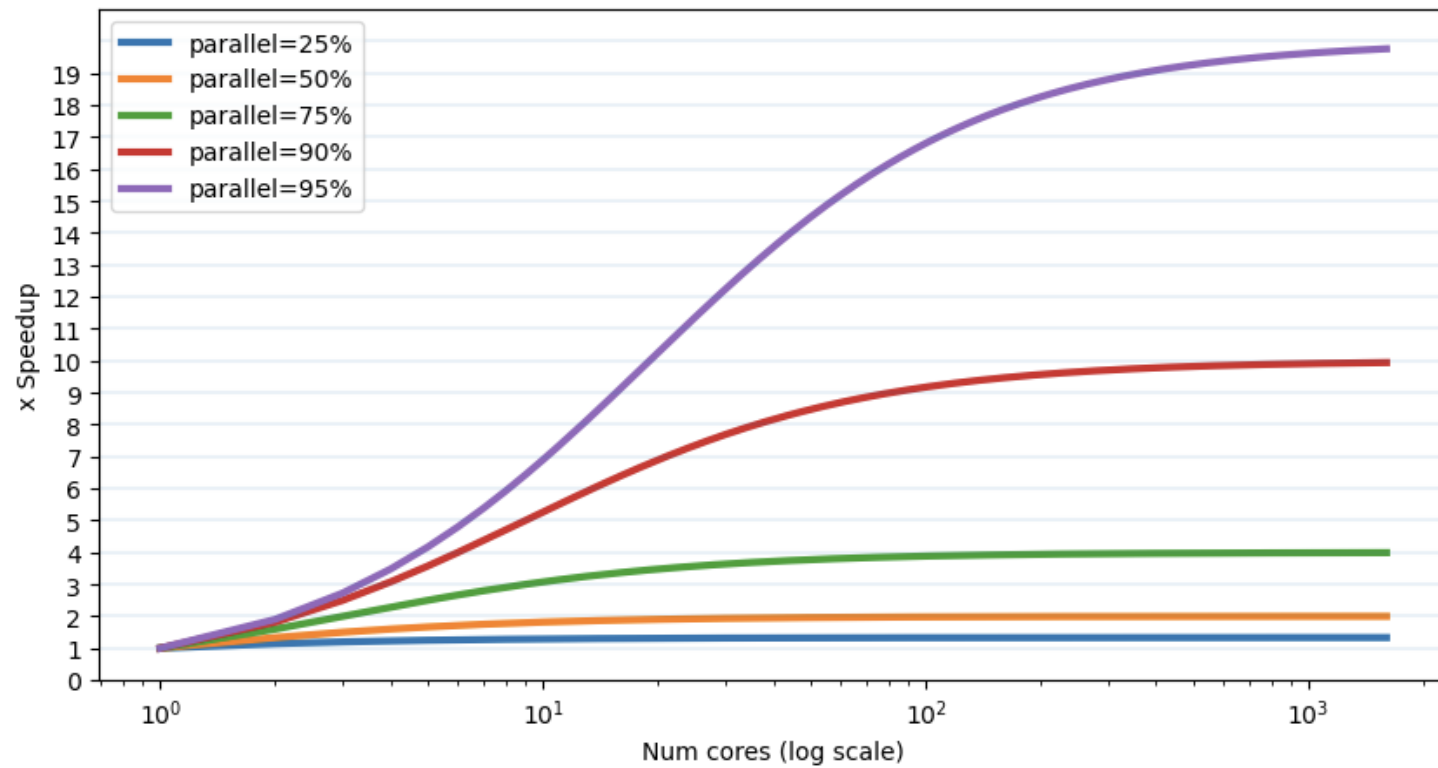


Amdahl's law





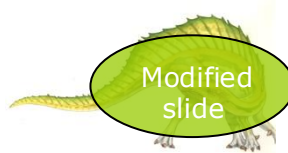
Amdahl's law





Pthreads

- A POSIX Threads standard (IEEE 1003.1c) API for thread creation and synchronization ... A ***Specification***, not ***implementation***
- API specifies behavior of the thread library, implementation is up to development of the library
- Common in UNIX operating systems (Linux & Mac OS X)





Pthreads Example

```
#include <pthread.h>
#include <stdio.h>

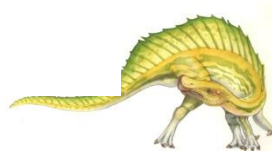
#include <stdlib.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */

    /* set the default attributes of the thread */
    pthread_attr_init(&attr);
    /* create the thread */
    pthread_create(&tid, &attr, runner, argv[1]);
    /* wait for the thread to exit */
    pthread_join(tid, NULL);

    printf("sum = %d\n", sum);
}
```





Pthreads Example (Cont.)

```
/* The thread will execute in this function */  
void *runner(void *param)  
{  
    int i, upper = atoi(param);  
    sum = 0;  
  
    for (i = 1; i <= upper; i++)  
        sum += i;  
  
    pthread_exit(0);  
}
```





Pthreads Code for Joining 10 Threads

```
#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
    pthread_join(workers[i], NULL);
```





Thread Cancellation

- Terminating a thread before it has finished
- Thread to be canceled is **target thread**
- Two general approaches: (via `pthread_setcancelstate()`)
 - **Asynchronous cancellation** terminates the target thread immediately
 - **Deferred cancellation** allows the target thread to periodically check if it should be cancelled
- Pthread code to create and cancel a thread:

```
pthread_t tid;

/* create the thread */
pthread_create(&tid, 0, worker, NULL);

. . .

/* cancel the thread */
pthread_cancel(tid);

/* wait for the thread to terminate */
pthread_join(tid, NULL);
```

